AI Engineering Syllabus: Beginner to Intermediate

Oct 29, 2026 · @Gyanendra Aryal

Duration: 24 weeks, about 8 hours per week **Prerequisite:** Basic comfort with Python and the command line **Method:** Each week has reading or video, hands-on work, a project tie-in using your repo, and a checkpoint you must pass before moving on.

Tools to set up in Week 1: Python 3.11+, Git, VS Code, Docker Desktop, Postman or curl, and free accounts at Groq, Qdrant Cloud, and Hugging Face.

Phase 1: Foundations (Weeks 1 to 4)

Week 1: How software runs

- What a computer has: CPU, RAM, disk, network
- Processes, threads, and workers; why Gunicorn runs workers
- HTTP methods, status codes (2xx, 4xx, 5xx), headers, JSON bodies
- Reading errors: 400, 404, 405, 500, 502, 504, and what each one blames
- Command line: cd, ls, grep, curl, environment variables

Project tie-in: Write a one-page explanation of your learn-vault 502. Name which component returned it and why.

Checkpoint: Explain the difference between a 400 and a 502 without notes.

Week 2: Python for real projects

- Functions, classes, modules, packages, imports, and sys.path
- Lists, dicts, sets, comprehensions, generators
- Exceptions, try/except, and logger.exception
- Global state and lazy loading (the _get_client() pattern in rag.py)

Project tie-in: Draw the call path of one chat request, from the view to generate_answer to the Groq call.

Checkpoint: Write a function that loads a config once and reuses it on every call.

Week 3: Virtual environments and dependencies

- venv, pip, requirements.txt vs pip freeze
- Why pip freeze dumps include transitive dependencies (the torch problem)
- Semantic versioning and pinning
- Environment variables and .env files; why secrets never go in Git

Project tie-in: Trim backend/requirements.txt to runtime-only dependencies and explain each line you kept.

Checkpoint: Build a fresh venv from the trimmed requirements and run the app in it.

Week 4: Math for embeddings and similarity

- Vectors and dimensions
- Dot product, Euclidean distance, cosine similarity
- Normalization, and why unit vectors make dot product equal cosine

Project tie-in: Re-run compare_embeddings.py and explain why cosine 1.0000 means your two embedders agree.

Resource: 3Blue1Brown, Essence of Linear Algebra (first 5 videos)

Checkpoint: Compute cosine similarity of two 3-dimensional vectors by hand and check it in Python.

Phase 2: Machine learning basics (Weeks 5 to 8)

Week 5: What a model is

- Inputs, weights, outputs, and loss
- Supervised, unsupervised, and self-supervised learning
- Training vs inference, and why your server only does inference
- Overfitting, underfitting, and the train/test split

Resource: Google Machine Learning Crash Course (free)

Checkpoint: Explain why a model can score well on training data and badly on new data.

Week 6: Model size, memory, and compute

- Parameters and RAM: roughly 2 bytes per parameter in 16-bit precision

- CPU vs GPU, and what CUDA is
- Latency vs throughput
- Why sentence-transformers pulled in torch and a much larger memory footprint than fastembed

Project tie-in: Estimate memory for three models you could run on Render's free tier, then check against the Metrics tab.

Checkpoint: Calculate the minimum RAM for a 7-billion-parameter model in 16-bit precision (about 14 GB before overhead).

Week 7: Neural networks in practice

- Layers, activations, and backpropagation (intuition, not derivation)
- Gradient descent and learning rate
- A small classifier with scikit-learn

Resource: 3Blue1Brown, Neural Networks series (first 4 videos); scikit-learn Getting Started guide

Checkpoint: Train a logistic regression on a public dataset and report accuracy on a held-out split.

Week 8: Evaluating models properly

- Precision, recall, F1, and when accuracy misleads
- Confusion matrices
- Simple baselines first

Project tie-in: Explain what evaluate_retrieval.py measures and what it misses.

Checkpoint: Build a labeled test set of 20 items and compute one metric on it.

Phase 3: Language models and transformers (Weeks 9 to 11)

Week 9: Tokens and text representation

- Tokenization (BPE and WordPiece at a high level)
- Why token count, not word count, drives cost and context limits
- Counting tokens with tiktoken or the provider's tokenizer

Checkpoint: Count tokens for three texts of different lengths and explain why the counts differ from word counts.

Week 10: The transformer

- Attention: queries, keys, values (intuition first)
- Multi-head attention, residual connections, layer norm, feed-forward blocks
- Encoder vs decoder models (BERT-style vs GPT-style)

Resource: Jay Alammar, The Illustrated Transformer (blog); Andrej Karpathy, Let's build GPT (YouTube, optional but strongly recommended)

Checkpoint: Draw the data flow through one transformer block from memory.

Week 11: How LLMs generate text

- Next-token prediction and sampling
- Temperature, top-p, top-k, and their effect on output
- Context window and why attention cost grows with length
- Hallucination: why models produce fluent wrong answers

Project tie-in: Run the same question at temperature 0 and 0.8, then explain why temperature 0.2 suits grounded answers.

Checkpoint: Explain in your own words why a model can confidently make things up.

Phase 4: Building AI applications (Weeks 12 to 16)

Week 12: Calling LLM APIs well

- Chat roles: system, user, assistant
- Parameters: max_tokens, temperature, timeout
- Streaming responses and why they reduce perceived latency
- Retries with backoff; errors 429, 500, and timeouts

Project tie-in: Add a retry wrapper around the Groq call in rag.py and a test that simulates a timeout.

Checkpoint: Make a call that fails on purpose and show your code returns a clean error instead of crashing.

Week 13: Prompt engineering

- Anatomy of a prompt: role, rules, context, format, examples
- Grounding: answer only from supplied sources
- Output formats: JSON mode, schemas, and parsing failures
- Testing prompts like code: fixed inputs, expected behavior, regression checks

Project tie-in: Rewrite the system prompt in rag.py. Test five questions before and after and record the differences.

Checkpoint: Show a prompt change that improved one case and broke another, and explain the tradeoff.

Week 14: Embeddings and chunking

- What embeddings capture and what they miss
- Chunk size and overlap
- Metadata (video ID, timestamps) must survive chunking
- Choosing an embedding model: dimension, speed, memory, language coverage

Project tie-in: Re-chunk one lecture at two sizes and compare which chunks retrieve best for five questions.

Checkpoint: Justify your chunk size with retrieval results, not intuition.

Week 15: Vector databases

- Collections, points, vectors, payloads, and filters
- HNSW indexes at a conceptual level; recall vs speed
- Top-k search and score thresholds
- Upserting, updating, and deleting points

Project tie-in: Write a script that counts points per video and flags any video with no chunks.

Checkpoint: Filter search results to one video ID and show the code.

Week 16: Retrieval-Augmented Generation (RAG) end to end

- Pipeline: ingest, chunk, embed, store, retrieve, generate, cite
- Score thresholds (MIN_SCORE) and the tradeoff between no answer and a wrong answer

- Hybrid search (keyword plus vector) and when it helps

Project tie-in: Diagram the learn-vault pipeline and mark where each failure from your debugging would occur.

Checkpoint: Trace one bad answer back to the stage that caused it: retrieval, prompt, or generation.

Phase 5: Evaluation, agents, and reliability (Weeks 17 to 19)

Week 17: Evaluating AI systems

- Retrieval metrics: recall@k, precision@k, MRR
- Generation metrics: faithfulness, relevance, correctness
- LLM-as-judge and its biases
- A golden dataset of 50+ questions with expected sources

Project tie-in: Build a 50-question golden set for learn-vault and record scores after every change.

Resource: Ragas documentation (RAG evaluation metrics)

Checkpoint: Show that a change improved one metric without hurting another.

Week 18: Tool use and agents

- Function calling with JSON schemas
- The loop: model requests a tool, you run it, you return the result
- Patterns: single tool, router, multi-step planning
- Guardrails: step limits, timeouts, allowed actions

Project tie-in: Add a tool that returns the library video list, so the model can answer which lectures cover serializers.

Resource: Anthropic and OpenAI function-calling guides; Hugging Face Agents course

Checkpoint: Build a two-tool agent with a hard limit of five steps and show it stopping correctly.

Week 19: Reliability, safety, and security

- Prompt injection: retrieved text trying to override instructions

- Keeping keys and private content out of logs and prompts
- Input validation and output filtering
- Rate limiting on public endpoints

Project tie-in: Test a transcript containing "ignore previous instructions" and confirm the answer still follows the rules.

Resource: OWASP Top 10 for LLM Applications

Checkpoint: Name three risks in your current app and one mitigation for each.

Phase 6: Production engineering (Weeks 20 to 22)

Week 20: Backend services and deployment

- REST API design: resources, status codes, error shapes
- Django and Django REST Framework request lifecycle
- Gunicorn workers, timeouts, and concurrency
- Docker images and containers
- Build-time vs runtime config (the Vite VITE_API_URL problem)

Project tie-in: Make a Dockerfile for the backend and build it locally.

Checkpoint: Run the container locally and call the chat endpoint with curl.

Week 21: Observability and performance

- Structured logging: latency, token counts, retrieval scores, error types
- Reading metrics: CPU, memory, request latency, error rate
- Timing each stage of a request to find bottlenecks
- Caching embeddings and answers

Project tie-in: Add timing logs to each stage of generate_answer and identify the slowest one.

Checkpoint: Show a log line that tells you where a slow request spent its time.

Week 22: Fine-tuning and model choices

- When to use prompting, RAG, or fine-tuning
- LoRA and parameter-efficient tuning at a conceptual level

- Dataset quality over quantity
- Open-weight vs hosted models: cost, privacy, control, latency

Resource: Hugging Face PEFT documentation; DeepLearning.AI, Finetuning Large Language Models

Checkpoint: Write a one-page decision memo arguing for or against fine-tuning learn-vault.

Phase 7: Capstone (Weeks 23 to 24)

Week 23: Build

Pick one project and ship it end to end:

- Upgrade learn-vault with streaming answers, citations in the UI, and a golden test set
- A course Q&A bot for a different subject, using the same pipeline
- A document assistant for a local business

Requirements: working deploy, golden evaluation set, logging, and a README explaining architecture and tradeoffs.

Week 24: Present and review

- Short design doc: problem, architecture, evaluation results, known failures
- A 5-minute recorded demo
- Review your own logs and metrics; list three things to fix next

Checkpoint: Someone who hasn't seen the project can run it from your README.

Glossary: learn these first

Term	Plain meaning
Embedding	A list of numbers representing the meaning of text
Dimension	How many numbers are in one embedding (384 for your model)
Vector database	A database that finds similar embeddings quickly

Term	Plain meaning
Inference	Running a trained model to produce output
Token	A chunk of text, roughly three quarters of a word on average
Context window	How much text a model can read in one request
Temperature	How random the output is; 0 is most deterministic
RAG	Retrieval-Augmented Generation: retrieve sources first, then answer
OOM	Out of memory: the process was killed for using too much RAM
Worker	A process that handles requests; Gunicorn manages them
Fine-tuning	Further training a model on your own data to change its behavior
Agent	A model that calls tools in a loop to complete a task

How to study this

- [ ] Work through one week at a time; don't start the next until you pass the checkpoint
- [ ] Keep a learning log: one entry per week with what you built, what broke, and why
- [ ] Use your repo for every project tie-in
- [ ] Break things on purpose: set a timeout too low, remove a dependency, send a bad request
- [ ] Re-run your evaluation after every change